Loup M. Marquez Luttringer – Project Manager

Matheus Lopez – Back-End & Front-End Developer

Noah Gibelli – AI Developer

Version: v1.0

Date: October 29, 2025

# Horus Project – Agent Implementation Justification

## (Using queuing for parallelization)

**Point 1 – A – Using queuing for parallelization**

Having efficient management of the different agents requires a very particular process. It's good to know that each agent will have a little time of waiting due to response generation. So, in this case, it's clear that we can't use a single queue to generate the answer to each task of all of the agents one after the other. First of all, we tried to use specific routes inside the REST API of the orchestrator to test the reaction of the API if several users tried to ask something at the same time. But we have seen that having an LLM generation can have an impact on the time of response for the other client and also if the agent crash, all of the process crash. So that is why we wanted to divide agents into several containers to counter this. But we had to think about the point that if an agent has already a task generation, it can be dangerous for the computer to set too many LLM calls at the same time. So, we decided to implement a distribution messaging process, with the help of RabbitMQ solution. The idea with this process was to parallelize task generation. Several agents can work at the same time. On this way, if an agent fail, the other agents will continue to work. It's a way to use smartly the memory of our machines. When an agent is occupied, it will put in the queue the task demand and will execute each task successively.

Each message includes metadata such as:

- Agent ID
- Task type
- Timestamp and correlation ID
- History of the previous agent calls

This design guarantees that no task is lost and that every action can be tracked — an essential property for both intelligence and accountability.

At a moment, we have tested the use of websockets process. It was a good solution that worked for our project, but the problem was with their management of them. In the future, we plan to have a big bunch of agents and using websocket can bring us some difficulties. First of all, we don't have a predefined structure like RabittMQ to manage them so it can be rising programming difficulty to have finally a low quality in terms of efficiency of the structure. Second, it consumes a lot of resources to maintain their activities. So, having websockets for the clients and for the agents can be a problem if you want to use small machines. It's not very optimized. Third, they can cause problem of security in front of attacks like Man-In-The-Middle (MITM) attacks and Denial-of-Service (DoS) attacks. Agents are the center of confidentiality so it's important to secure the process with a big solution. The main point also is that it supports reliable message queues, retries, and independent consumers.